

ARTIFICIAL INTELLIGENCE AND KNOWLEDGE ENGINEERING

REPORT – EXERCISE 2

Álvaro Puebla Ruisánchez

Contents

1	<u>Introduction and Goal</u>	2
2	<u>Theoretical Background</u>	3
2.1	Two-Player Zero-Sum Games	3
2.2	Minimax Algorithm	3
2.3	Alpha-Beta Pruning	3
3	<u>Formal Problem Formulation</u>	5
3.1	Game State	5
3.2	Legal Moves	5
3.3	State Transition	5
3.4	Decision Tree	5
3.5	Constraints	6
4	<u>Heuristic Evaluation Functions</u>	7
4.1	H1 - Piece Count (<code>piece</code>)	7
4.2	H2 - Cumulative Advancement (<code>advance</code>)	7
4.3	H3 - Proximity Threat (<code>threat</code>)	7
4.4	H4 - Column Centrality (<code>central</code>)	7
4.5	H5 - Weighted Combination (<code>combined</code>)	7
5	<u>Implementation</u>	9
5.1	Board Representation	9
5.2	Move Generation (<code>get_moves</code>)	9
5.3	State Transition (<code>apply_move</code>)	9
5.4	Minimax with Alpha-Beta	10
5.5	Extended Mode and Agent Protocol	11
5.6	Adaptive Strategy	11
6	<u>How to Run the Program</u>	12
6.1	Board Input	12
7	<u>Worked Example</u>	13
8	<u>Libraries Used</u>	14
9	<u>Conclusion and Implementation Issues</u>	15
9.1	Summary	15
9.2	Issues Encountered	15

1 Introduction and Goal

The goal of this exercise is to implement the two-player board game *Breakthrough* using the Minimax decision algorithm with alpha-beta pruning. The program accepts a board from a file supplied with `-board FILE` or uses the built-in starting position via `-default-board`. It also supports multiple heuristic evaluation functions and allows two independent agents to play against each other with different configurations.

Breakthrough was designed by Dan Troyka (2000) [8]. Its simple rules but rich strategy make it an ideal testbed for classic game-tree search algorithms.

2 Theoretical Background

2.1 Two-Player Zero-Sum Games

A two-player zero-sum game is a tuple $\langle S, A, T, u \rangle$ where S is a set of states, A is a set of actions, $T : S \times A \rightarrow S$ is a deterministic transition function, and $u : S \rightarrow \mathbb{R}$ is a utility function such that the gain of one player equals the loss of the other [4].

Breakthrough satisfies all these conditions: information is perfect and complete, moves are deterministic, and exactly one player wins (utility +1 for the winner, -1 for the loser).

2.2 Minimax Algorithm

Let s denote the current state, d the remaining search depth, and P the player to move. The minimax value $V(s, d)$ is defined recursively:

$$V(s, d) = \begin{cases} u(s) & \text{if } s \text{ is terminal or } d = 0 \\ \max_{a \in A(s)} V(T(s, a), d - 1) & \text{if } P = \text{MAX} \\ \min_{a \in A(s)} V(T(s, a), d - 1) & \text{if } P = \text{MIN} \end{cases} \quad (1)$$

When the depth limit is reached before a terminal state, a heuristic function $h : S \rightarrow \mathbb{R}$ approximates the position's value.

2.3 Alpha-Beta Pruning

Alpha-beta pruning [3] maintains two running bounds: α (best score MAX can guarantee, initially $-\infty$) and β (best score MIN can guarantee, initially $+\infty$). A branch is pruned whenever $\alpha \geq \beta$:

- **β -cutoff** at a MAX node: the MIN ancestor will not allow a value exceeding β .
- **α -cutoff** at a MIN node: the MAX ancestor already has a guaranteed value of at least α .

With perfect move ordering the effective branching factor drops from b to $\sqrt{b}$, doubling the reachable depth for the same budget [3].

The pseudocode matching the actual implementation is given below.

Algorithm 1 Minimax with Alpha-Beta Pruning (as implemented)

```

1: function MINIMAX( $s, d, \alpha, \beta, maximizing, me, opp, h$ )
2:    $w \leftarrow \text{CHECKWINNER}(s)$ 
3:   if  $w = me$  then return (WIN +  $d$ , None)
4:   else if  $w = opp$  then return (LOSS -  $d$ , None)
5:   end if
6:    $current \leftarrow me$  if  $maximizing$  else  $opp$ 
7:    $M \leftarrow \text{GETMOVES}(s, current)$ 
8:   if  $d = 0$  or  $M = \emptyset$  then return ( $h(s, me)$ , None)
9:   end if
10:   $best \leftarrow \text{None}$ 
11:  if  $maximizing$  then
12:     $v \leftarrow -\infty$ 
13:    for  $m \in M$  do
14:       $score, \_ \leftarrow \text{MINIMAX}(\text{APPLYMOVE}(s, m), d-1, \alpha, \beta, \text{false}, me, opp, h)$ 
15:      if  $score > v$  then
16:         $v \leftarrow score$ 
17:         $best \leftarrow m$ 
18:      end if
19:       $\alpha \leftarrow \max(\alpha, v)$ 
20:      if  $\alpha \geq \beta$  then
21:        break ▷  $\beta$ -cutoff
22:      end if
23:    end for
24:  else
25:     $v \leftarrow +\infty$ 
26:    for  $m \in M$  do
27:       $score, \_ \leftarrow \text{MINIMAX}(\text{APPLYMOVE}(s, m), d-1, \alpha, \beta, \text{true}, me, opp, h)$ 
28:      if  $score < v$  then
29:         $v \leftarrow score$ 
30:         $best \leftarrow m$ 
31:      end if
32:       $\beta \leftarrow \min(\beta, v)$ 
33:      if  $\alpha \geq \beta$  then
34:        break ▷  $\alpha$ -cutoff
35:      end if
36:    end for
37:  end if return ( $v$ ,  $best$ )
38: end function

```

3 Formal Problem Formulation

3.1 Game State

A **game state** is a tuple $\sigma = \langle G, P \rangle$ where:

- $G \in \{B, W, _, o\}^{m \times n}$ is the board - an $m \times n$ matrix of cell values.
- $P \in \{B, W\}$ is the player whose turn it is to move.

In the default configuration $m = n = 8$. The cell symbols are: B (first player's piece), W (second player's piece), $_$ (empty), and o (the square vacated by the most recent move - treated as empty for move generation).

Initial state: Player B occupies rows 0 and 1; player W occupies rows $m-2$ and $m-1$. All remaining cells are empty. **White (W) moves first.**

Terminal states:

$$\text{Win for } B : \exists c \in \{0, \dots, n-1\} \text{ s.t. } G[m-1][c] = B \quad (2)$$

$$\text{Win for } W : \exists c \in \{0, \dots, n-1\} \text{ s.t. } G[0][c] = W \quad (3)$$

3.2 Legal Moves

Let $dr = +1$ for player B and $dr = -1$ for player W . From a piece at (r, c) the target squares (r', c') are:

1. **Straight forward:** $(r', c') = (r + dr, c)$, provided $G[r'][c'] \in \{_, o\}$ (empty or last-move marker).
2. **Diagonal forward:** $(r', c') = (r + dr, c \pm 1)$, provided $0 \leq c \pm 1 < n$ and $G[r'][c'] \neq P$ (empty, last-move marker, or occupied by the opponent - i.e. capturing is allowed diagonally).

3.3 State Transition

Applying move $m = ((r, c) \rightarrow (r', c'))$:

1. A fresh copy of the board is made.
2. All cells equal to o are reset to $_$.
3. $G[r][c] \leftarrow o$; $G[r'][c'] \leftarrow P$ (capturing any opponent piece there).
4. The active player toggles: $P \leftarrow \bar{P}$.

3.4 Decision Tree

The decision tree is rooted at the current state σ_0 . Nodes at even depth are MAX nodes (the agent to optimise); nodes at odd depth are MIN nodes (the opponent). Leaves are states at depth $d_{\max}$ or terminal states. On the default 8×8 board, the effective branching factor is approximately 30-35 in the opening.

3.5 Constraints

1. The board size $n \times m$ is a parameter; default is 8×8 .
2. The maximum search depth d is a parameter (CLI flag `-d`).
3. Pieces may only move *forward* (toward the opponent's back rows).
4. Straight-ahead capture is forbidden; diagonal capture is the only way to remove an opponent piece.
5. The player who first places a piece on the opposite back row wins.

4 Heuristic Evaluation Functions

All five heuristics are implemented in the file `breakthrough.py` and are selected via the CLI flag `-H` (for player B) and `-H2` (for player W). A positive score favours the perspective player.

4.1 H1 - Piece Count (piece)

$$h_1(\sigma, P) = |\text{own pieces}| - |\text{opponent pieces}|$$

Rewards captures and penalises losses. Fast to compute but ignores position.

4.2 H2 - Cumulative Advancement (advance)

$$h_2(\sigma, B) = \sum_{(r,c): G[r][c]=B} r - \sum_{(r,c): G[r][c]=W} (m-1-r)$$

Each piece contributes its distance from its own starting edge, encouraging forward progress.

4.3 H3 - Proximity Threat (threat)

$$h_3(\sigma, B) = \sum_{(r,c): G[r][c]=B} \max(0, r - \lfloor m/2 \rfloor) - \sum_{(r,c): G[r][c]=W} \max(0, (m-1-r) - \lfloor m/2 \rfloor)$$

Pieces that have crossed the midline receive a bonus proportional to their remaining distance to the goal. This aggressively rewards pieces close to winning.

4.4 H4 - Column Centrality (central)

$$h_4(\sigma, B) = \sum_{(r,c): G[r][c]=B} \left(\lfloor n/2 \rfloor - |c - (n-1)/2| \right) - \sum_{(r,c): G[r][c]=W} (\dots)$$

Central columns provide more diagonal attack and evasion options. This heuristic gives a small bonus to centrally placed pieces.

4.5 H5 - Weighted Combination (combined)

$$h_5 = 10 \cdot h_1 + 3 \cdot h_2 + 5 \cdot h_3 + 1 \cdot h_4$$

A linear combination of all four heuristics. Weights were chosen empirically: piece count matters most (10), followed by threats (5), advancement (3), and centrality (1). This is the **default heuristic**.

Table 1: Summary of heuristic evaluation functions.

ID	Name	Key idea	CLI key
H1	Piece count	Maximise army size	<code>piece</code>
H2	Advancement	Maximise forward progress	<code>advance</code>
H3	Threat	Reward pieces near the goal	<code>threat</code>
H4	Centrality	Prefer central columns	<code>central</code>
H5	Combined	Weighted sum of H1-H4	<code>combined</code>

5 Implementation

5.1 Board Representation

The board is a Python `List[List[str]]`. Each cell holds one of the four symbols B, W, _, o. Copying is trivial:

Listing 1: Board copy - $O(mn)$.

```
1 def board_copy(board):
2     return [row[:] for row in board]
```

5.2 Move Generation (get_moves)

The function iterates over all cells, identifies the current player's pieces, and validates up to three target squares per piece (straight, left diagonal, right diagonal). `is_free` treats both _ and o as empty.

Listing 2: Move generation - straight and diagonal.

```
1 def get_moves(board, player):
2     m, n = len(board), len(board[0])
3     dr = 1 if player == B else -1
4     opponent = OPP[player]
5     moves = []
6     for r in range(m):
7         for c in range(n):
8             if board[r][c] != player:
9                 continue
10            nr = r + dr
11            if not (0 <= nr < m):
12                continue
13            if is_free(board[nr][c]):           # straight forward
14                moves.append(((r, c), (nr, c)))
15            for dc in (-1, 1):                 # diagonals
16                nc = c + dc
17                if 0 <= nc < n:
18                    target = board[nr][nc]
19                    if is_free(target) or target == opponent:
20                        moves.append(((r, c), (nr, nc)))
21     return moves
```

5.3 State Transition (apply_move)

The board is never modified in place. `apply_move` always returns a new board, clearing old o markers before setting the new one.

Listing 3: Immutable state transition.

```
1 def apply_move(board, move, player):
2     board = board_copy(board)
3     (r1, c1), (r2, c2) = move
4     for r in range(len(board)):
5         for c in range(len(board[0])):
6             if board[r][c] == LAST:
7                 board[r][c] = EMPTY
```

```

8     board[r1][c1] = LAST
9     board[r2][c2] = player
10    return board

```

5.4 Minimax with Alpha-Beta

The recursive minimax function directly implements Algorithm 1. A global counter `_nodes_visited` is incremented at every call and reported on `stderr` at the end.

Listing 4: Core minimax with alpha-beta pruning.

```

1  _nodes_visited = 0
2
3  def minimax(board, depth, alpha, beta, maximizing, me, opp, hfn):
4      global _nodes_visited
5      _nodes_visited += 1
6
7      w = check_winner(board)
8      if w == me:
9          return WIN + depth, None    # faster win = higher score
10     if w == opp:
11         return LOSS - depth, None   # faster loss = lower score
12
13     current = me if maximizing else opp
14     moves = get_moves(board, current)
15
16     if depth == 0 or not moves:
17         return hfn(board, me), None
18
19     best_move = None
20     if maximizing:
21         v = float('-inf')
22         for mv in moves:
23             child = apply_move(board, mv, current)
24             score, _ = minimax(child, depth-1, alpha, beta, False, me,
25                               opp, hfn)
26             if score > v:
27                 v, best_move = score, mv
28             alpha = max(alpha, v)
29             if alpha >= beta:
30                 break                # beta-cutoff
31         return v, best_move
32     else:
33         v = float('inf')
34         for mv in moves:
35             child = apply_move(board, mv, current)
36             score, _ = minimax(child, depth-1, alpha, beta, True, me,
37                               opp, hfn)
38             if score < v:
39                 v, best_move = score, mv
40             beta = min(beta, v)
41             if alpha >= beta:
42                 break                # alpha-cutoff
43         return v, best_move

```

5.5 Extended Mode and Agent Protocol

The program supports two extended modes:

Single-process extended mode: The flags `-d2` and `-H2` set a different depth and heuristic for player *W*. Both agents run within the same Python process.

Listing 5: Two agents with different strategies, single process.

```
python3 breakthrough.py --default-board -d 4 -H combined -d2 3 -H2
advance
```

Two-process agent mode: The flag `-agent B/W` makes the program load a board from `-board FILE`, compute exactly one move, write the updated board, and exit. An external orchestrator can coordinate two such invocations by reusing the board file between turns:

Listing 6: File-based orchestration of two independent agents.

```
python3 breakthrough.py --agent B -d 4 -H combined --board board.txt
python3 breakthrough.py --agent W -d 3 -H advance --board board.txt
```

5.6 Adaptive Strategy

The combined heuristic **H5** adapts implicitly: the threat bonus grows as pieces approach the goal, naturally shifting the agent from piece-count-oriented play in the opening toward a rush strategy in the endgame - without requiring any explicit phase detection.

6 How to Run the Program

All modes combine a board source with AI settings and optional extras. The board source must be either the default starting position or an explicit board file supplied with `-board FILE`.

Listing 7: Quick-start examples.

```
# AI vs AI (default board, depth 3, combined heuristic)
python3 breakthrough.py --default-board -d 3 -H combined

# Watch AI play step by step
python3 breakthrough.py --default-board -d 3 -H combined --visual --step

# Human (White) vs AI (Black)
python3 breakthrough.py --default-board --human W -d 3 -H combined

# Two agents with different configs
python3 breakthrough.py --default-board -d 4 -H combined -d2 3 -H2
  advance

# Agent mode (compute one move for B)
python3 breakthrough.py --agent B -d 4 -H combined --board board.txt

# Custom board from file
python3 breakthrough.py --board board.txt -d 3 -H combined
```

6.1 Board Input

Table 2: Supported board input modes.

Option	Arguments	Description
<code>-default-board</code>	none	Uses the built-in 8×8 starting position. The board dimensions may be adjusted with <code>-rows</code> and <code>-cols</code> .
<code>-board FILE</code>	FILE	Loads the board from the specified file. This is the only file-based custom-board mode supported by the program.

Sample output written to `stdout` and `stderr` after a game:

```
=====
GAME OVER
=====
  a b c d e f g h
8 - - - - - - -
7 - - - - - - -
...
1 - - - - - - -
Rounds played: 31 | Winner: B

Nodes visited : 1842503      <- stderr
Execution time: 4.2731 s    <- stderr
```

7 Worked Example

Consider a simplified 4×4 board after several moves, shown in Figure 1. It is Black's turn.

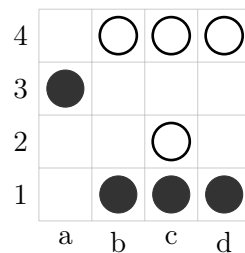


Figure 1: Example 4×4 board state (Black ● to move). Black has pieces on b1, c1, d1, a3; White ○ has pieces on b4, c4, d4, c2.

With search depth $d = 2$:

1. Black generates moves from all its pieces. The piece at a3 can move straight to a4 (which is the last row) — an immediate win.
2. Minimax evaluates this branch: the resulting state is terminal with Black winning, returning score $\text{WIN} + 2 = 1\,000\,002$.
3. All other branches return lower scores. The algorithm selects $a3 \rightarrow a4$.

The depth bonus $+d$ (remaining depth) is key: a win found closer to the root scores *higher*, so the agent always picks the **fastest** available win.

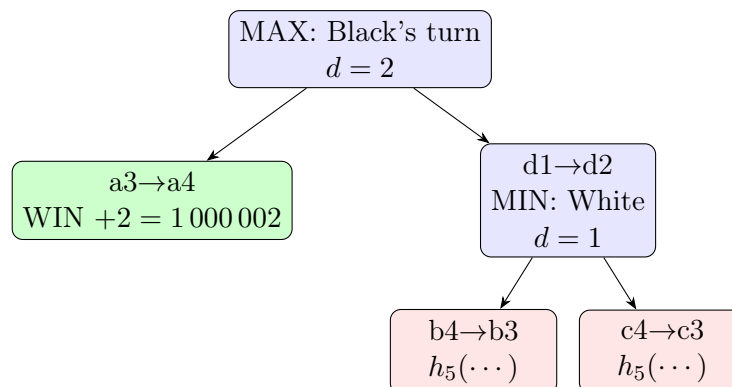


Figure 2: Partial decision tree (depth 2). The winning branch is selected immediately.

8 Libraries Used

The implementation relies exclusively on Python's standard library - no third-party packages are required:

- `sys` - writing to `stderr` and exiting cleanly.
- `time` - wall-clock timing for performance reporting.
- `argparse` - command-line argument parsing (all flags described in Section 6 are handled here).

The code has been tested with Python 3.10 and later.

9 Conclusion and Implementation Issues

9.1 Summary

The program correctly implements the full Breakthrough game with: legal move generation (including capture semantics and the o marker), five heuristic evaluation functions (H1-H5), minimax search with alpha-beta pruning, basic mode (both players share depth and heuristic) and extended mode (independent agents with configurable `-d2/-H2` flags), human vs. AI mode, visual step-by-step playback, and `-agent` one-shot mode for file-based orchestration. Node count and execution time are reported to `stderr` after every run.

9.2 Issues Encountered

1. **Board immutability.** Early versions mutated the board in place and used an undo stack. This caused subtle bugs when move generation read partially-modified state. Switching to copy-on-write semantics (`apply_move` always returns a new object) eliminated the entire bug class at a modest memory cost.
2. **The o marker.** The assignment requires marking the source square of the last move with o. Initially, `get_moves` treated o as occupied, incorrectly blocking straight-forward moves back to that square. The fix was to make `is_free` return `True` for both `_` and o.
3. **Win-score depth bonus sign.** The bonus must be applied to *remaining* depth, not elapsed depth. Higher remaining depth means the win is found closer to the root - correctly preferring faster wins. An earlier version added depth to losses too, accidentally making the agent *seek* faster losses.
4. **Branching factor and depth.** At depth 5 on an 8×8 board the node count can exceed 10^7 , causing runtimes of tens of seconds. Depth 3-4 gives the best balance of play quality and speed. Move ordering (e.g. placing captures first) could further improve alpha-beta efficiency but was not implemented.
5. **Extended-mode protocol.** Designing a clean two-process protocol required careful coordination of the evolving board file and the `Winner:` detection line. The `-agent` flag ensures the process exits after a single move, making it straightforward to compose with any external orchestrator.

References

- [1] C. E. Shannon, *Programming a Computer for Playing Chess*, Philosophical Magazine, Ser. 7, Vol. 41, No. 314, 1950.
- [2] J. von Neumann and O. Morgenstern, *Theory of Games and Economic Behavior*, Princeton University Press, 1944.
- [3] D. E. Knuth and R. W. Moore, *An Analysis of Alpha-Beta Pruning*, Artificial Intelligence, Vol. 6, No. 4, pp. 293-326, 1975.
- [4] N. Nisan, T. Roughgarden, E. Tardos, and V. V. Vazirani (Eds.), *Algorithmic Game Theory*, Cambridge University Press, 2007.
- [5] M. Maschler, E. Solan, and S. Zamir, *Game Theory*, Cambridge University Press, 2013.
- [6] M. J. Osborne, *An Introduction to Game Theory*, Oxford University Press, 2004.
- [7] J. F. Nordström, *Introduction to Game Theory*, Lecture notes, course materials.
- [8] Wikipedia contributors, *Breakthrough (board game)*, Wikipedia, The Free Encyclopedia, [https://en.wikipedia.org/wiki/Breakthrough_\(board_game\)](https://en.wikipedia.org/wiki/Breakthrough_(board_game)), accessed March 2026.
- [9] *Breakthrough - opis zasad* (Rules description), course materials.